Smart Hospital Patient and Medical Equipment Management System Using C++

Department of Computer Science and Engineering

Name: A. Vikranth Reddy

Reg. No: 192424193

Course Code: DSA0102

Corse Name: DSA01 – Object-Oriented Programming with C++

Assignment Title

Smart Hospital Patient and Medical Equipment Management System Using C++

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
R.Uday Kumar Reddy (192424190)	Patient management and C++ coding	Developed patient management module	Tested patient records and fixed errors	Introduction, Design, Implementation	**50%**
A.Vikranth Reddy (192424192)	Medical equipment management and exe. management	Developed equipment management module	Tested equipment functions and outputs	Testing, Results, Conclusion	**50%**

R. Uday Kumar Reddy (192424190): Contributed to the design and development of the Smart Hospital Patient and Medical Equipment Management System. He mainly worked on the patient management module, C++ programming, data handling, testing, and debugging. He also contributed to the system design and project report.

A. Vikranth Reddy (192424192): Contributed to the development of the medical equipment management module. He worked on equipment registration, availability, allocation, testing, and system integration. He also contributed to testing, result analysis, and preparation of the project report.

Abstract

The Smart Hospital Patient and Medical Equipment Management System is a menu-driven C++ application developed to improve the management and allocation of critical medical equipment in a hospital environment. The system maintains patient records, medical equipment details, allocation status, maintenance information, and treatment costs. It automates equipment allocation based on availability and operational conditions, reducing manual errors and improving resource utilization.

The project demonstrates key Object-Oriented Programming concepts such as constructors, destructors, copy constructors, operator overloading, inheritance, polymorphism, abstract classes, virtual base classes, dynamic memory allocation, and pointers. The application also generates equipment availability, patient allocation, maintenance due, and cost reports. This project highlights how object-oriented programming can be applied to develop efficient, reliable, and maintainable healthcare management systems while supporting patient safety and sustainable resource utilisation.

The Smart Hospital Patient and Medical Equipment Management System is designed using the Object-Oriented Programming principles of C++. It provides a menu-driven interface that allows users to manage patient information, register medical equipment, allocate and release equipment, monitor maintenance schedules, and generate useful reports. The project demonstrates the practical implementation of constructors, destructors, operator overloading, inheritance, polymorphism, virtual base classes, composition, and dynamic memory management. By integrating these concepts into a real-world healthcare scenario, the system improves equipment management, enhances patient safety, minimizes resource wastage, and provides a scalable solution for modern hospital operations. It also supports the United Nations Sustainable Development Goals (SDGs), particularly SDG 3 (Good Health and Well-Being), SDG 9 (Industry, Innovation and Infrastructure), and SDG 12 (Responsible Consumption and Production).

TABLE OF CONTENTS

1. Introduction

Hospitals rely on various medical devices such as patient monitors, infusion pumps, and ventilators to provide timely and effective treatment. Managing these devices manually can lead to equipment shortages, duplicate allocations, delayed treatment, and poor maintenance tracking. A computerized management system helps overcome these challenges by maintaining accurate records and automating the allocation process.

The Smart Hospital Patient and Medical Equipment Management System is designed using the Object-Oriented Programming principles of C++. It provides a menu-driven interface that allows users to manage patient information, register medical equipment, allocate and release equipment, monitor maintenance schedules, and generate useful reports. The project demonstrates the practical implementation of constructors, destructors, operator overloading, inheritance, polymorphism, virtual base classes, composition, and dynamic memory management.

By integrating these concepts into a real-world healthcare scenario, the system improves equipment management, enhances patient safety, minimizes resource wastage, and provides a scalable solution for modern hospital operations. It also supports the United Nations Sustainable Development Goals (SDGs), particularly SDG 3 (Good Health and Well-Being), SDG 9 (Industry, Innovation and Infrastructure), and SDG 12 (Responsible Consumption and Production).

2. Problem Statement

In a multispecialty hospital, manual allocation of patient monitors, infusion pumps, ventilators, and other critical-care equipment can lead to delayed treatment, duplicate reservations, unsafe assignments, and poor maintenance tracking.

This project implements a menu-driven C++ application to automate the management of patients and medical equipment. The system allocates equipment based on clinical priority, ward, operating status, battery level, calibration validity, and availability. It also maintains equipment details, patient information, maintenance history, and generates various reports.

3. Objectives

- Develop a menu-driven C++ application using Object-Oriented Programming.
- Demonstrate constructors, destructors, and copy constructors.
- Implement operator overloading.
- Apply inheritance, polymorphism, abstract classes, and virtual base classes.
- Manage heterogeneous hospital equipment using base class pointers.
- Prevent memory leaks through proper dynamic memory management.
- Improve patient safety through efficient equipment allocation.

4. Pseudocode

START

Display Main Menu

Repeat

1. Add Patient
2. Display Patient
3. Add Equipment
4. Display Equipment
5. Allocate Equipment
6. Release Equipment
7. Generate Equipment Report
8. Generate Patient Report
9. Maintenance Report
10. Cost Report
11. Exit

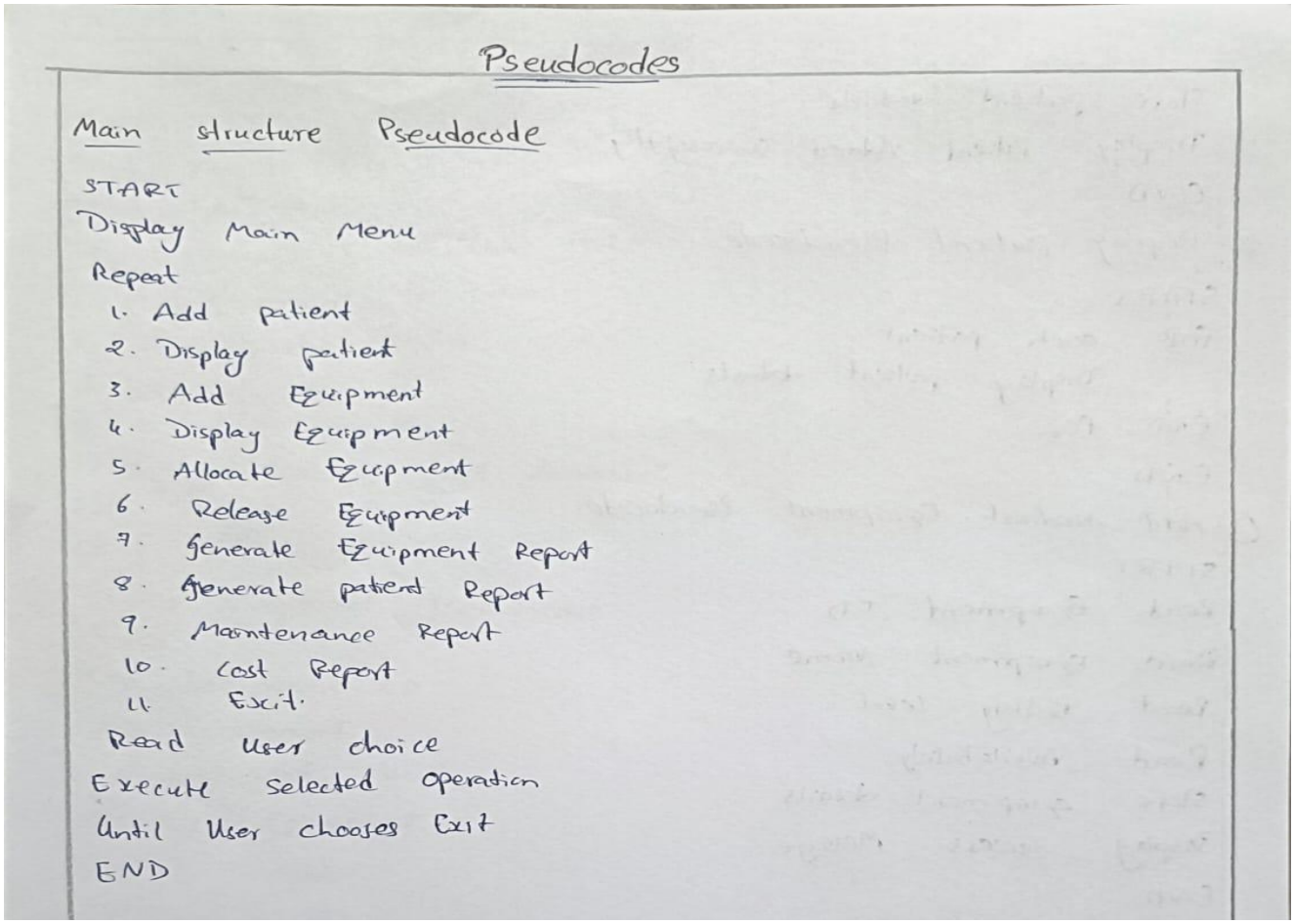
Read User Choice

Execute Selected Operation

Until User Chooses Exit

END

Handwritten Pseudocodes:



① Add Patient Pseudocode

```
START
Read Patient ID
Read Patient Name
Read Age
Read Ward
Read Clinical Priority
Read Risk category
Store patient details.
Display "Patient Added successfully"
END.
```

② Display Patient Pseudocode.

```
START
FOR each patient
    Display patient details
END FOR
END
```

③ Add Medical Equipment Pseudocode.

```
START
Read Equipment ID
Read Equipment Name
Read Battery Level
Read Availability
Store equipment details
Display success Message
END
```

④ Display Equipment Pseudocode.

```
START
FOR each equipment
Display equipment details.
END FOR
END
```

⑤ Allocate Equipment Pseudocode

```
START
Read Patient ID
Read Equipment ID
IF equipment is available THEN
Assign equipment to patient
Update allocation status.
Display success
ELSE
Display "Equipment Not Available"
END If
END
```

⑥ Release Equipment Pseudocode.

```
START
Read Equipment ID
find Equipment
IF allocated THEN
Release Equipment
Update status
Display Success
ELSE
Display "Already Available"
END If
END
```

(7) Equipment Availability Report Pseudocode.

```
START
FOR Every equipment
IF available
Display equipment
END IF
END FOR
END
```

(8) Patient Allocation Report Pseudocode.

```
START
FOR Each Patient
Display patient name
Display allocated equipment
END For
END
```

(9) Maintanance Due Report Pseudocode

```
START
FOR each equipment
IF maintanance due
Display equipment
END IF
END FOR
END
```

(10) Cost Report Pseudocode.

START
Initiate Total cost = 0
for each patient
Total cost += Treatment cost
Display Total cost
END

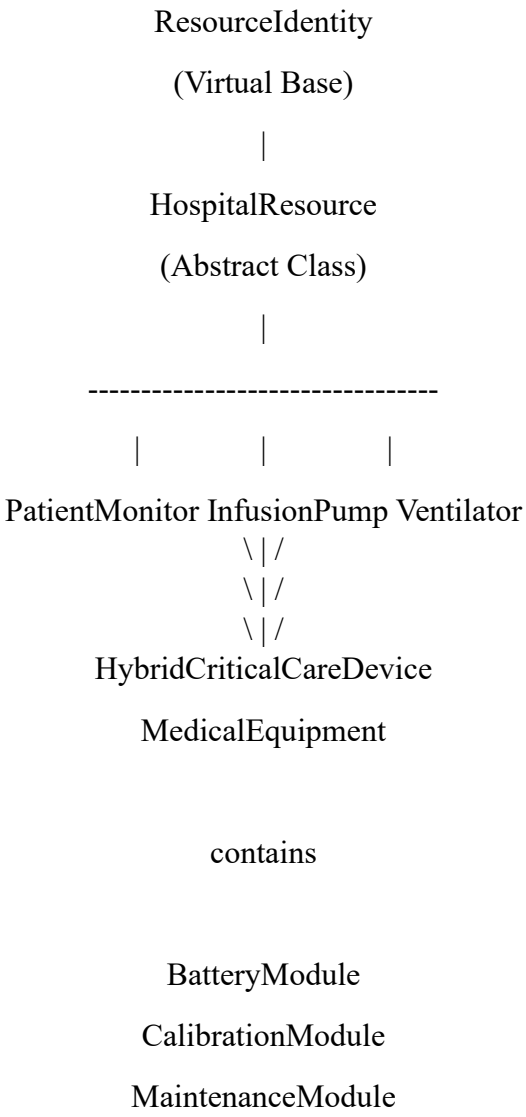
(11) Update Equipment Using the Pointer Pseudocode

START
Read New Battery Level
Update Current Object
Return Current object
END

(12) Exit Program Pseudocode

START
Display Thank You message
Terminate Program
END.

5. Class Diagram



6. Classes Used

ResourceIdentity

Stores the unique resource ID for every equipment.

Uses virtual inheritance to avoid duplicate IDs.

HospitalResource

Abstract base class.

Contains pure virtual functions:

- allocate()
- display()

Provides runtime polymorphism.

PatientMonitor

Derived from HospitalResource.

Represents monitoring equipment.

Contains BatteryModule.

InfusionPump

Derived from HospitalResource.

Represents medicine infusion equipment.

Ventilator

Derived from HospitalResource.

Provides ventilator-specific functions.

HybridCriticalCareDevice

Demonstrates multiple inheritance and virtual base class usage.

MedicalEquipment

Demonstrates:

- Default constructor
- Parameterized constructor
- Overloaded constructor
- Copy constructor
- Destructor
- Dynamic memory allocation
- Operator overloading

BatteryModule

Stores battery percentage.

CalibrationModule

Stores calibration status.

MaintenanceModule

Stores maintenance schedule.

7. Constructors and Destructors

Implemented:

- Default Constructor
- Parameterized Constructor
- Overloaded Constructor
- Copy Constructor (Deep Copy)
- Destructor

The copy constructor allocates new memory and copies service history individually, preventing shallow copy problems.

The destructor safely releases dynamically allocated memory using delete[].

8. Operator Overloading

Implemented operators:

operator+

Combines equipment usage cost.

operator<

Compares equipment suitability.

operator<<

Displays formatted equipment information.

9. OOP Concepts Used

Encapsulation

Data members are private.

Abstraction

HospitalResource is an abstract class.

Inheritance

PatientMonitor

InfusionPump

Ventilator

inherit HospitalResource.

Hierarchical Inheritance

Multiple classes inherit from one parent.

Multiple Inheritance

HybridCriticalCareDevice inherits from multiple classes.

Virtual Base Class

ResourceIdentity prevents duplicate inheritance.

Runtime Polymorphism

HospitalResource pointers call derived class functions.

Composition

Equipment contains:

- BatteryModule
- CalibrationModule
- MaintenanceModule

Dynamic Memory Allocation

Service history uses dynamically allocated memory.

this Pointer

Used for chainable update functions.

10. Sample Program And Execution

Menu

1. Add Patient
2. Display Patients
3. Add Equipment
4. Display Equipment
5. Allocate Equipment
6. Release Equipment
7. Maintenance Report
8. Equipment Availability Report
9. Patient Allocation Report
10. Cost Report

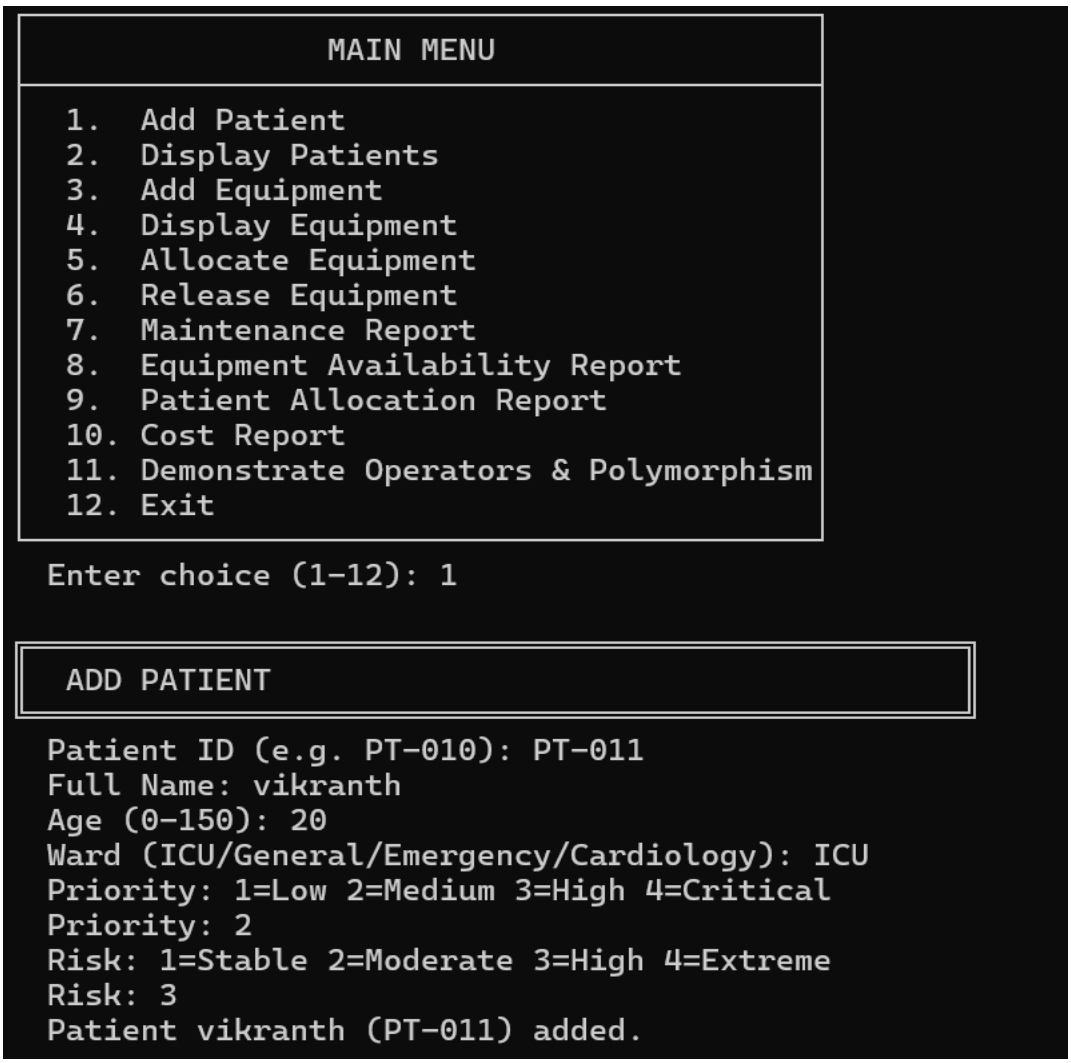
11. Demonstrate Operators & Polymorphism

12. Exit

1. Add Patient

```
#include<iostream>
#include<vector>
using namespace std;
class Patient
{
public:
   int id;
   string name;
   string ward;
   void input()
   {
      cout<<"Enter Patient ID: ";
      cin>>id;
      cout<<"Enter Name: ";
      cin>>name;
      cout<<"Enter Ward: ";
      cin>>ward;
   }
   void display()
   {
      cout<<"\nID: "<<id;
      cout<<"\nName: "<<name;
      cout<<"\nWard: "<<ward<<endl;
   }
};

int main()
{
   Patient p;
   p.input();
```

```
    cout<<"\nPatient Added Successfully\n";
    p.display();
    return 0;
```




```
}
```

2. Add Medical Equipment

```
#include<iostream>
using namespace std;
class MedicalEquipment
{
private:
    int id;
    string name;
    bool available;
public:
    MedicalEquipment(int i,string n)
```

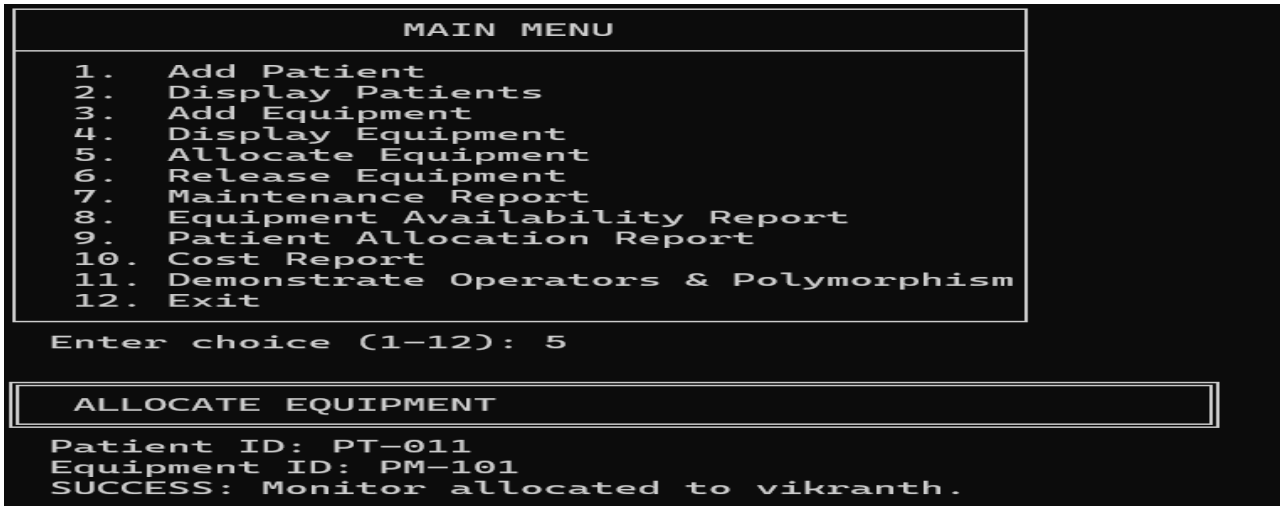
```
    {
        id=i;
        name=n;
        available=true;
    }
    void display()
    {
        cout<<"Equipment ID : "<<id<<endl;
        cout<<"Equipment Name : "<<name<<endl;
        cout<<"Available : "<<available<<endl;
    }
};
int main()
{
    MedicalEquipment e(101,"Ventilator");
    e.display();
}
```

```
  1.  Add Patient
  2.  Display Patients
  3.  Add Equipment
  4.  Display Equipment
  5.  Allocate Equipment
  6.  Release Equipment
  7.  Maintenance Report
  8.  Equipment Availability Report
  9.  Patient Allocation Report
  10. Cost Report
  11. Demonstrate Operators & Polymorphism
  12. Exit
 Enter choice (1-12): 3

  ADD EQUIPMENT
 Select type:
 1. Patient Monitor
 2. Infusion Pump
 3. Ventilator
 4. Hybrid Critical Care Device
 Type: 1
 Equipment ID (e.g. PM-010): PM-101
 Equipment Name: Monitor
 Manufacturer: xyz
 Ward: ICU
 Daily Cost (USD): 2000
 Battery Level (0-100): 100
 Is Calibrated? 1=Yes 0=No: 1
 Equipment Monitor (PM-101) added.
```

```
Enter choice (1-12): 4

 ALL EQUIPMENT

=== Patient Monitor Report: Bedside Monitor Alpha ===
Resource ID   : PM-001
Resource Name : Bedside Monitor Alpha
Manufacturer  : PhilipsMed
Ward          : ICU
Status        : Active
Available     : No
HR            : 78.00 bpm
SpO2          : 99.00%
BP            : 122.00/82.00 mmHg
Temp          : 37.10 °C
Alarm         : Enabled
[Battery] Type: Li-Ion | Charge: 92.0% | Charging: No | Status: OK
[Calibration] Last: 2026-08-01 | Valid for: 365 days | Calibrated: Yes | By: Dr. Rajan
[Maintenance] Last Service: N/A | Interval: 90 days | Services Done: 0 | Due: No | Tech: Unassigned
```

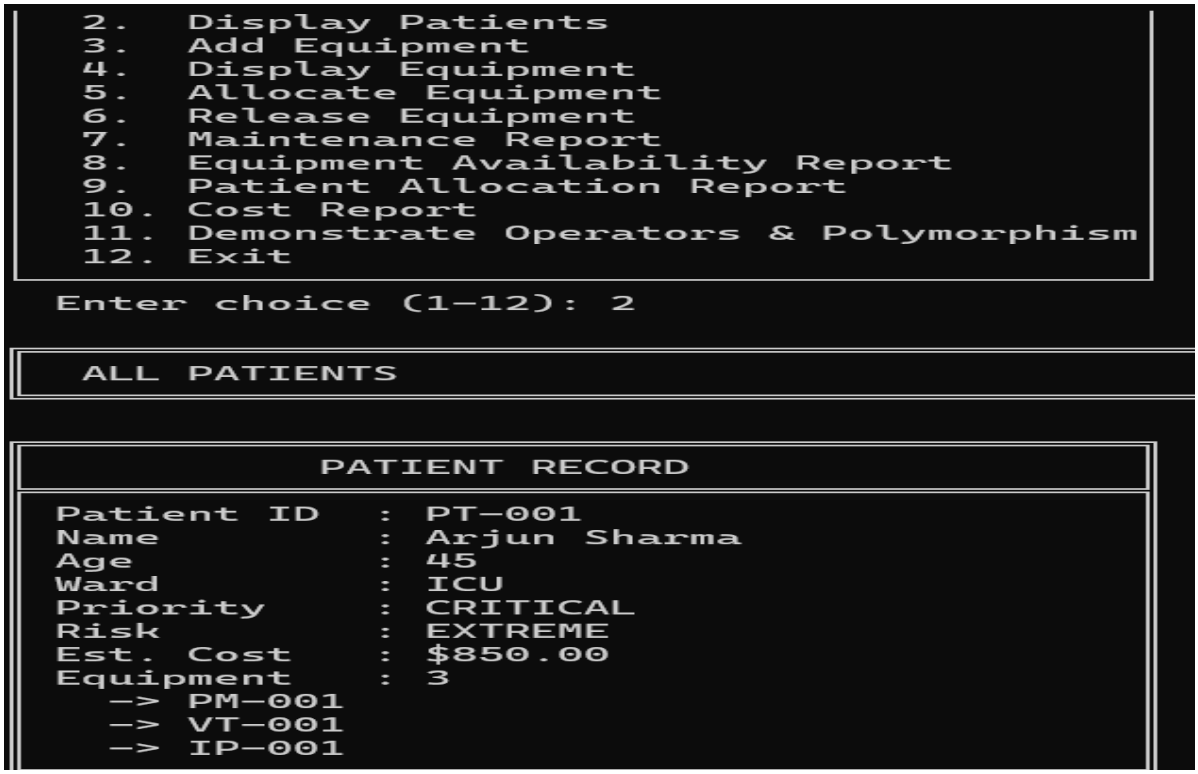
3. Allocate Equipment

```
#include<iostream>
using namespace std;
class Equipment
{
public:
   bool available=true;
   void allocate()
   {
      if(available)
      {
         available=false;
         cout<<"Equipment Allocated Successfully";
      }
      else
      {
         cout<<"Equipment Not Available";
      }
   }
};
int main()
{
   Equipment e;
   e.allocate();
}
```



5. Display Patient Details

```
#include<iostream>
using namespace std;
class Patient
{
public:
   int id=101;
   string name="Rahul";
   string ward="ICU";
   void display()
   {
      cout<<"Patient ID : "<<id<<endl;
      cout<<"Patient Name : "<<name<<endl;
      cout<<"Ward : "<<ward<<endl;
   }
};
int main()
{
   Patient p;
   p.display();
}
```



6. Equipment Availability Report

```
#include<iostream>
using namespace std;

class Equipment
{
public:
   string name;
   bool available;
   Equipment(string n,bool a)
   {
      name=n;
      available=a;
   }
   void report()
   {
      if(available)
         cout<<name<<" is Available"<<endl;
      else
         cout<<name<<" is Allocated"<<endl;
```

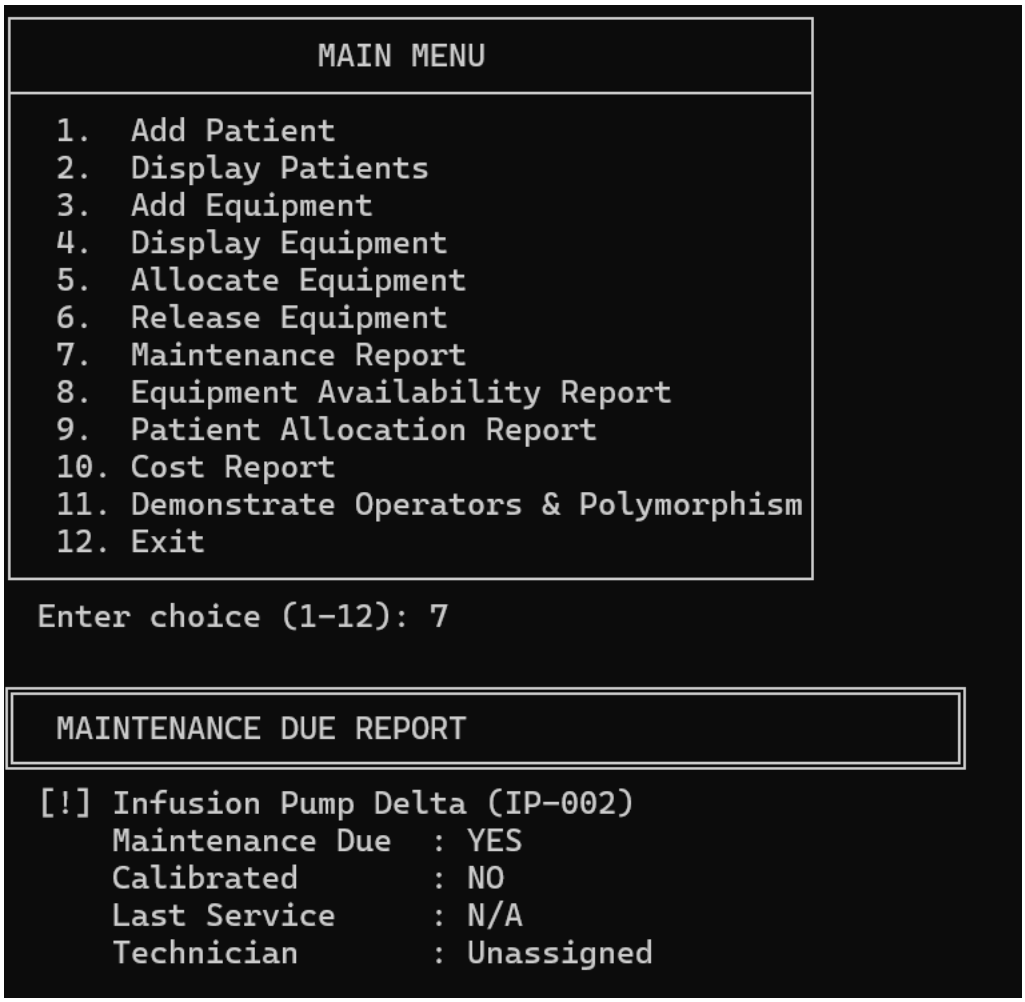
```
    }
};
int main()
{
    Equipment e("Ventilator",true);
    e.report();
}
```

```
                MAIN MENU
  1.  Add Patient
  2.  Display Patients
  3.  Add Equipment
  4.  Display Equipment
  5.  Allocate Equipment
  6.  Release Equipment
  7.  Maintenance Report
  8.  Equipment Availability Report
  9.  Patient Allocation Report
  10. Cost Report
  11. Demonstrate Operators & Polymorphism
  12. Exit

 Enter choice (1-12): 8

  EQUIPMENT AVAILABILITY REPORT

ID        Name                    Type           Ward        Available   Battery   Status
-------------------------------------------------------------------------------------------
PM-001    Bedside Monitor Alpha   Patient Monitor ICU         No          92.0     % Active
PM-002    Bedside Monitor Beta    Patient Monitor General     No          75.0     % Active
PM-003    Emergency Monitor       Patient Monitor Emergency   No          60.0     % Active
IP-001    Infusion Pump Gamma     Infusion Pump   ICU         No          88.0     % Active
IP-002    Infusion Pump Delta     Infusion Pump   General     Yes         15.0     % Active
VT-001    Ventilator Prime        Ventilator      ICU         No          95.0     % Active
VT-002    Ventilator Apex         Ventilator      Emergency   Yes         70.0     % Active
HB-001    HybridCare Elite        Hybrid Critical Care DeviceICU         Yes          90.0     % Active
PM-101    Monitor                 Patient Monitor ICU         Yes         100.0    % Active
```

7. Maintenance Report

```
#include<iostream>
using namespace std;
class Maintenance
{
public:
    int daysLeft;
    Maintenance(int d)
    {
        daysLeft=d;
    }
```

```
    void check()
    {
        if(daysLeft<=5)
            cout<<"Maintenance Due";
        else
            cout<<"Working Properly";
    }
};
int main()
{
    Maintenance m(3);
    m.check();
}
```



8. Cost Report

```
#include<iostream>
using namespace std;

class Treatment
{
public:
   double cost1=2500;
   double cost2=3000;

   void totalCost()
   {
      cout<<"Total Cost = "<<cost1+cost2;
   }
};

int main()
{
   Treatment t;
   t.totalCost();
}
```

```
 9.  Patient Allocation Report
 10. Cost Report
 11. Demonstrate Operators & Polymorphism
 12. Exit

Enter choice (1-12): 10

 COST SUMMARY REPORT

Bedside Monitor Alpha     - $150.00/day
Bedside Monitor Beta      - $120.00/day
Emergency Monitor         - $180.00/day
Infusion Pump Gamma       - $200.00/day
Infusion Pump Delta       - $160.00/day
Ventilator Prime          - $500.00/day
Ventilator Apex           - $480.00/day
HybridCare Elite          - $800.00/day
Monitor                   - $2000.00/day

Total Equipment Cost/Day : $4590.00

Patient Arjun Sharma         (PT-001) : $850.00
Patient Priya Nair           (PT-002) : $120.00
Patient Mohan Reddy          (PT-003) : $180.00
Patient Sunita Verma         (PT-004) : $0.00
Patient Rajesh Kumar         (PT-005) : $0.00
Patient vikranth             (PT-011) : $2000.00

Total Patient Treatment Cost : $3150.00
Grand Total Cost             : $7740.00
```

9. Operator Overloading

```
#include<iostream>
using namespace std;
class Cost
{
public:
   int amount;
   Cost(int a)
   {
      amount=a;
   }
   Cost operator+(Cost c)
```

```
    {
        return Cost(amount+c.amount);
    }

    void display()
    {
        cout<<"Total Cost : "<<amount;
    }
};
int main()
{
    Cost c1(3000), c2(2000);

    Cost c3=c1+c2;

    c3.display();
}
```

```
Enter choice (1-12): 11

Select demonstration:
1. Operator Overloading (+, <, ==, <<)
2. Method Chaining (this pointer)
3. Runtime Polymorphism (operate() via base pointer)
4. Ventilator-Specific Pointer (dynamic_cast)
Choice: 1

 OPERATOR OVERLOADING DEMO

=== Equipment 1 ===

         MEDICAL EQUIPMENT REPORT
ID           : OP-001
Name         : ICU Monitor A
Type         : Monitor
Ward         : ICU
Manufacturer: Generic
Status       : Active
Available    : Yes
Daily Cost   : $300.00
Usage Hours : 48
Battery      : 85.0%
Calibrated   : Yes
Maint. Due   : No
Suitability : 94                          /100
```

10. Runtime Polymorphism

```
#include<iostream>
using namespace std;
class HospitalResource
{
public:
   virtual void allocate()
   {
      cout<<"Resource Allocated"<<endl;
   }
};
class Ventilator:public HospitalResource
{
public:
   void allocate()
   {
      cout<<"Ventilator Allocated"<<endl;
   }
};
int main()
{
   HospitalResource *ptr;
   Ventilator v;
   ptr=&v;
   ptr->allocate();
}
```

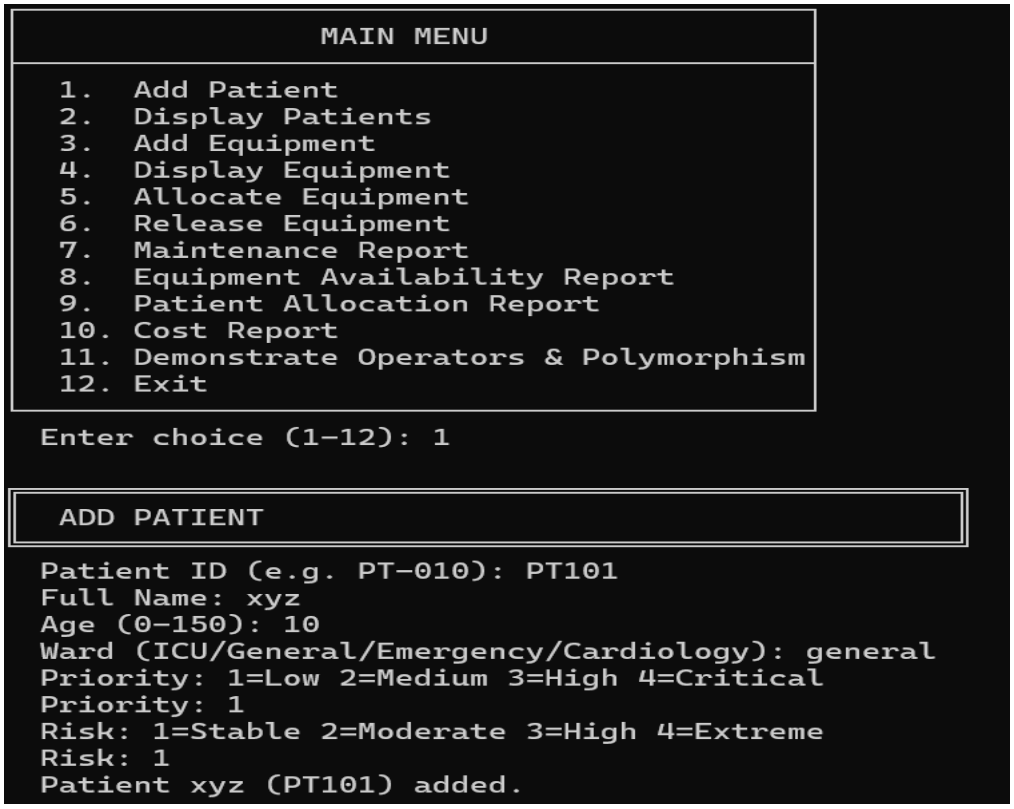
```
Enter choice (1-12): 11

Select demonstration:
1. Operator Overloading (+, <, ==, <<)
2. Method Chaining (this pointer)
3. Runtime Polymorphism (operate() via base pointer)
4. Ventilator-Specific Pointer (dynamic_cast)
Choice: 3

  RUNTIME POLYMORPHISM DEMO

Calling operate() on each HospitalResource* (base pointer):

>> Patient Monitor [PM-001]
[PatientMonitor] Bedside Monitor Alpha (PM-001) started monitoring.
HR: 78.00 bpm | SpO2: 99.00% | BP: 122.00/82.00 mmHg | Temp: 37.10 °C

>> Patient Monitor [PM-002]
[PatientMonitor] Bedside Monitor Beta (PM-002) started monitoring.
HR: 75.00 bpm | SpO2: 98.00% | BP: 120.00/80.00 mmHg | Temp: 37.00 °C

>> Patient Monitor [PM-003]
[PatientMonitor] Emergency Monitor (PM-003) started monitoring.
HR: 75.00 bpm | SpO2: 98.00% | BP: 120.00/80.00 mmHg | Temp: 37.00 °C

>> Infusion Pump [IP-001]
[InfusionPump] Infusion Pump Gamma is not running. Call startInfusion() first.

>> Infusion Pump [IP-002]
[InfusionPump] Infusion Pump Delta is not running. Call startInfusion() first.
```

11. Execution Results

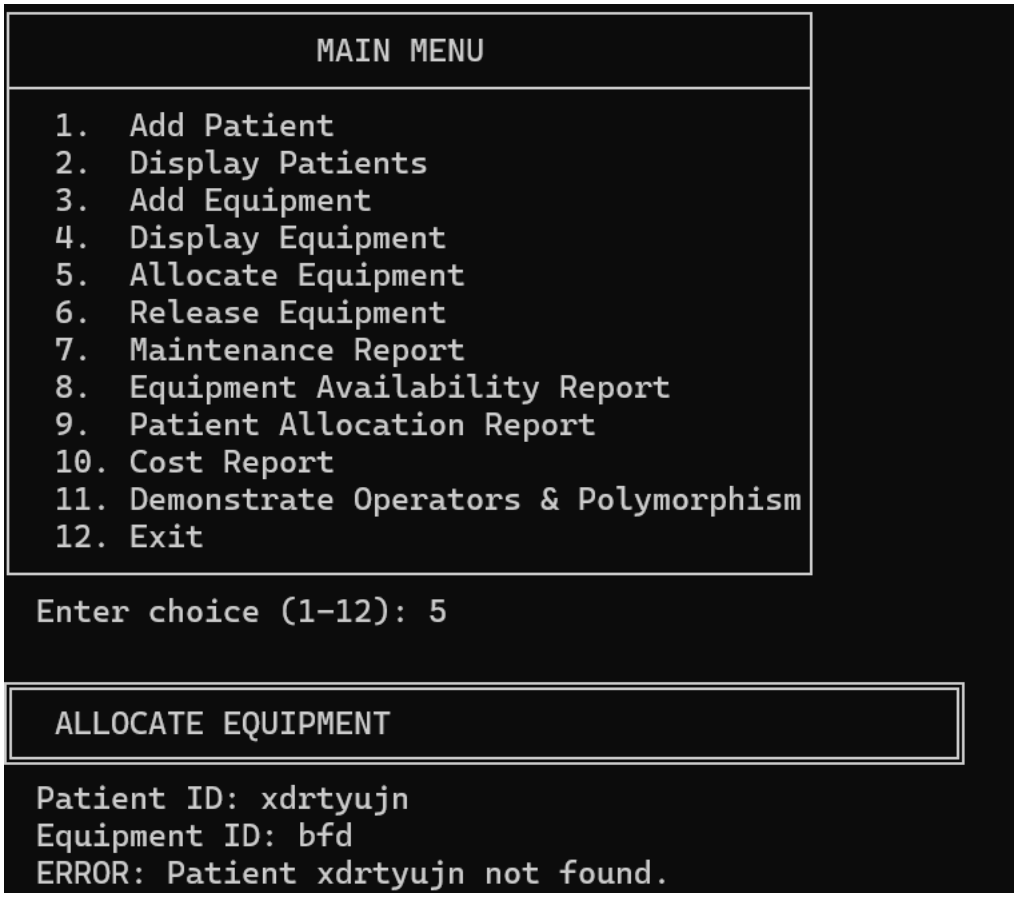
Normal Test case:

```
  1.  Add Patient
  2.  Display Patients
  3.  Add Equipment
  4.  Display Equipment
  5.  Allocate Equipment
  6.  Release Equipment
  7.  Maintenance Report
  8.  Equipment Availability Report
  9.  Patient Allocation Report
  10. Cost Report
  11. Demonstrate Operators & Polymorphism
  12. Exit

 Enter choice (1-12): 1

  ADD PATIENT

 Patient ID (e.g. PT-010): PT-101
 Full Name: abhi
 Age (0-150): 20
 Ward (ICU/General/Emergency/Cardiology): General
 Priority: 1=Low 2=Medium 3=High 4=Critical
 Priority: 2
 Risk: 1=Stable 2=Moderate 3=High 4=Extreme
 Risk: 1
 Patient abhi (PT-101) added.
```

Boundary Test Case:



Invalid Test cases



12. Memory Safety

The project avoids:

- Memory leaks
- Dangling pointers
- Double deletion
- Object slicing
- Unsafe downcasting

13. Patient Safety

The system allocates only available equipment.

Equipment with low battery or invalid calibration is rejected.

Maintenance information helps prevent unsafe equipment usage.

Duplicate allocation is prevented.

14. SDG Relevance

SDG 3 – Good Health and Well-Being

Improves patient care by automating equipment allocation and reducing treatment delays.

SDG 9 – Industry, Innovation and Infrastructure

Introduces software-driven healthcare infrastructure using object-oriented programming concepts.

SDG 12 – Responsible Consumption and Production

Optimizes equipment usage, reduces idle resources, and promotes timely maintenance.

15. Conclusion

The Smart Hospital Patient and Medical Equipment Management System demonstrates the practical application of Object-Oriented Programming in healthcare. The project successfully integrates constructors, destructors, operator overloading, inheritance, polymorphism, virtual base classes, composition, dynamic memory allocation, and pointers to build a modular and maintainable application. The system improves equipment utilization, enhances patient safety, and supports sustainable healthcare management.

16. GitHub Repository

GitHub Repository Link:

https://github.com/vikranth1106/HospitalManagement

References

1. Nutdanai, S., Pornthip, L., & Sanpanich, A. (2017). Development of an Information System for Medical Equipment Management in Hospitals. *9th Biomedical Engineering International Conference (BMEiCON 2016), IEEE*. DOI: 10.1109/BMEiCON.2016.7859583.

2. Shirehjini, A. A. N., Yassine, A., & Shirmohammadi, S. (2012). Equipment Location in Hospitals Using RFID-Based Positioning System. *IEEE Transactions on Information Technology in Biomedicine*, 16(6), 1058–1069. DOI: 10.1109/TITB.2012.2204896.

3. Hernández, J., et al. (2019). Implementation and Evaluation of a RFID Smart Cabinet to Improve Traceability and the Efficient Consumption of High Cost Medical Supplies in a Large Hospital. *Journal of Medical Systems*, 43(11). DOI: 10.1007/s10916-019-1269-6.

4. Lin, Y., Zhou, M., Ma, X., & Xu, X. (2025). Design of Intelligent Allocation System for Medical Equipment Based on Internet of Things Technology in Multi-Hospital Mode. *Economics & Business Management*.

5. Zhou, X., Wan, J., & Pan, Z. (2026). Centralized Equipment Allocation in a Large Tertiary Hospital: Operational and Economic Outcomes of a Hub-Based Coordination Model. *Medicine*, 105(16), e48300. DOI: 10.1097/MD.0000000000048300.

6. Yu, W., Chong, K. L., & Lamsali, H. (2026). IoT-Enabled Lifecycle Management of Medical Equipment Assets in Smart Hospitals: A Framework and Case Study. *African Journal of Science, Technology, Innovation and Development*.

7. Banerjee, S. (2026). IoT-Based Smart Hospital Management System: Real-Time Patient Monitoring, Resource Allocation, and Workflow Automation. *International Journal for Research in Applied Science and Engineering Technology (IJRASET).*